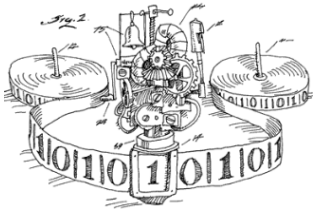
Teoría de la Computación y Lenguajes Formales



Teoría de la Computación y Lenguajes Formales: Sesión 7. El Motor Intérprete Genérico de Autómatas (Hito 1)

Édgar Chacón Ramírez
Departamento de Computación
Semestre 2026 - B

Alcance de la sesión 7

UNIVERSIDAD DE LOS ANDES VENEZUELA

La Arquitectura Genérica

Especificación en YAML

Lógica del Intérprete

Bucle de Simulación

Conclusión

UNIVERSIDAD DE LOS ANDES VENEZUELA

De la Diapositiva al Código: ¿Por qué un Motor Genérico?

El problema del código duro (Hardcoding)

UNIVERSIDAD DE LOS ANDES VENEZUELA

La inconveniencia práctica

En la Semana 3 programamos el recurso directamente en la lógica del código (con sentencias *switch* o *if − else*).
Si el proceso cambia, o si queremos simular una máquina diferente, nos veríamos obligados a modificar y recompilar el software.

La solución: El Intérprete

Separamos el **comportamiento del proceso** (los datos en un archivo YAML) del **motor de ejecución** (el código Python). El motor es único y genérico; es capaz de leer y ejecutar cualquier descripción de autómata.

UNIVERSIDAD DE LOS ANDES VENEZUELA

La Especificación de la Planta: Estructurando el archivo YAML

Estructura del YAML del Recurso

UNIVERSIDAD DE LOS ANDES VENEZUELA

Cabecera y Estados Marcados

```yaml
recurso: "Estacion_Pintura"
  tipo: "Maquina_De_Pintura"
  estado_inicial: "OCIOSO"
  estados_finales:
    exito:
      - "COMPLETADO"
    falla:
      - "FALLA_CRITICA"
```

Eventos y transiciones

```yaml
eventos:
  - id: "cargar"
    controlable: true
  - id: "detectar_falla"
    controlable: false

transiciones:
  - de: "OCIOSO"
    evento: "cargar"
    a: "LISTO"
```

UNIVERSIDAD DE LOS ANDES VENEZUELA

Lógica del Intérprete en Python: Inicialización y Carga

Carga de la Especificación en Memoria

UNIVERSIDAD DE LOS ANDES VENEZUELA

Lectura y Mapeo Dinámico

Usamos 'pyyaml' para cargar los datos y mapear la lista de transiciones en un diccionario de fácil acceso para el motor:

```
import yaml

# Cargar archivo YAML
with open("recurso.yaml", "r", encoding="utf-8") as f:
    spec = yaml.safe_load(f)

# Mapear transiciones: transitions_map[de][evento] = a
transitions_map = {}
for t in spec['transiciones']:
    transitions_map.setdefault(t['de'], {})[t['evento']] = t['a']

# Obtener alfabeto y estados finales
alphabet = [e['id'] for e in spec['eventos']]
final_states = spec['estados_finales']['exito'] + spec['estados_finales']['falla
```

La Clase DFA Genérica

Estructura y Atributos de Ejecución

La clase 'DFA' recibe los datos parseados y mantiene el estado actual en memoria:

```
class DFA:
    def __init__(self, states, alphabet, transitions, q0, F):
        self.states = states
        self.alphabet = alphabet
        self.transitions = transitions
        self.current_state = q0
        self.F = F
        self.history = []
    def transition(self, event):
        if event not in self.alphabet:
            raise ValueError(f"Evento '{event}' fuera del alfabeto.")
        state_transitions = self.transitions.get(self.current_state, {})
        if event in state_transitions:
            self.current_state = state_transitions[event]
            self.history.append(event)
        else:
            raise ValueError(f"Transicion no permitida en estado '{self.current_
```

UNIVERSIDAD DE LOS ANDES VENEZUELA

Simulación Interactiva: El Bucle de Consola

El Bucle de Ejecución en Consola

UNIVERSIDAD DE LOS ANDES VENEZUELA

Entrada de eventos por parte del usuario

```
# Instanciar el motor
recurso_dfa = DFA(alphabet, transitions_map, spec['estado_inicial'], final_states

while True:
  try:
    evento = input(f"[{recurso_dfa.current_state}] Evento -> ").strip()
    if evento.lower() == 'salir':
      break
    recurso_dfa.transition(evento)

    # Verificar hitos
    if recurso_dfa.current_state in spec['estados_finales']['exito']:
      print("Hito alcanzado! EXITO.")
    elif recurso_dfa.current_state in spec['estados_finales']['falla']:
      print("Alerta! FALLA CRITICA.")

  except ValueError as err:
    print(err)
```

Auditoría y Aceptación de la Traza

UNIVERSIDAD DE LOS ANDES VENEZUELA

Verificación final al terminar

Al salir de la simulación, el motor determina formalmente si la traza completa (historial de eventos del proceso) es aceptada por el lenguaje del autómata:

```
print(f"Traza final: {'->'.join(recurso_dfa.history)}")
print(f"Estado final del recurso: {recurso_dfa.current_state}")

if recurso_dfa.current_state in recurso_dfa.F:
    print("Resultado: Traza de eventos ACEPTADA por el automata.")
else:
    print("Resultado: Traza RECHAZADA (proceso incompleto).")
```

Conclusión de la Sesión 7

UNIVERSIDAD DE LOS ANDES VENEZUELA

Logros del Hito 1

- **Generalidad:** Logramos que un único programa sea capaz de ejecutar cualquier proceso lineal secuencial local con solo cargar su archivo YAML.
- **Ontología y Código:** El archivo YAML representa la ontología descriptiva, mientras que el script Python actúa como el motor de ejecución.
- **Verificación:** El motor evalúa automáticamente la validez de la traza de auditoría respecto a las reglas sintácticas del autómata.

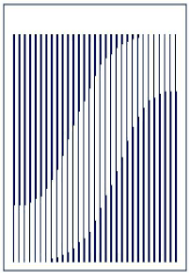
Próxima sesión

Estudiaremos las **limitaciones de los Autómatas Finitos** y daremos inicio al **Bloque 3 (Semana 7)**: Introducción a la Recursividad, Sub-procesos y los **Autómatas de Pila (PDA)**.

¡Muchas gracias!



UNIVERSIDAD
DE LOS ANDES
VENEZUELA



Édgar Chacón Ramírez
email: echacon@ula.ve